INSTITUT NATIONAL DES SCIENCES APPLIQUÉES ROUEN NORMANDIE

Département ITI 3

Rapport de réalisation

Projet Intégratif ITInéraire

Étudiants

GRONDIN	Yann
ASSI	Hadi
LESTERLIN	Maximilien
ROLLA	Benoît
YVANOFF	Nicolas

Projet GitLab

4 juin 2026

Table des matières

1 Introduction **2**

2 Architecture générale **3**
2.1 Matériel utilisé . . . 3
2.2 Organisation logicielle . . . 3
2.3 Réalisation d'une acquisition . . . 4

3 Partie 1 : Estimation de la trajectoire **5**
3.1 Problématiques de la partie 1 . . . 5
3.2 Capteurs retenus . . . 5
3.3 Calibration des capteurs . . . 6
3.4 Script d'acquisition . . . 6
3.5 Pré-traitement et filtrage des données brutes . . . 6
3.6 Modélisation du filtre de Kalman . . . 7
3.7 Résultats obtenus . . . 9
3.8 Conclusion partie 1 . . . 12

4 Partie 2 : Fonctionnalité supplémentaire **13**
4.1 Présentation de la fonctionnalité choisie . . . 13
4.2 Cahier des charges de la fonctionnalité . . . 13
4.3 Choix des capteurs . . . 13
4.4 Architecture logicielle . . . 14
4.5 Traitement des données . . . 15
4.6 Analyse des données . . . 16
4.7 Résultats . . . 19
4.8 Perspectives d'amélioration . . . 21

5 Partie bonus et acquisitions **22**
5.1 Conclusion . . . 23

1 Introduction

Le projet ITInéraire consiste à concevoir un ITIPhone, un outil capable de faire un suivi de position précis, si possible comparable ou meilleur que Strava. Pour cela, nous disposons de multiples capteurs dont un GPS (avec le système GNSS) et une IMU qui seront les principaux capteurs pour atteindre cet objectif. La problématique plus générale de ce projet est pour nous d'apprendre à manipuler des flux de données de capteurs plus ou moins bruités afin d'obtenir une représentation cohérente du mouvement, que l'on peut comparer à une référence comme Strava.

Ce rapport détaillera d'abord l'architecture générale du projet, les choix techniques qui ont été faits et l'organisation globale des traitements. Ensuite, nous détaillerons la manière dont l'estimation de la trajectoire a été implémentée ainsi que les traitements en post-acquisition qui ont été réalisés. Puis nous expliquerons notre fonctionnalité supplémentaire, un compteur de squats et la manière dont elle a été intégrée dans notre ITIPhone. Nous présenterons globalement les acquisitions que nous avons réalisées et les résultats. Finalement, nous décrirons la répartition du travail, et nous conclurons sur ce que l'on a appris de ce projet et sur les résultats obtenus. Et nous discuterons enfin des perspectives d'amélioration de nos résultats.

2 Architecture générale

2.1 Matériel utilisé

Pour mener à bien ce projet nous avons utilisé plusieurs capteurs et composants à notre disposition :

- GNSS DFRobot TEL0157, nous donnant des mesures GNSS précises provenant de multiples systèmes de satellites
- IMU MinIMU-9 v6 Gyro, qui combine un accéléromètre, un gyroscope et un magnétomètre en un seul composant
- Raspberry Pi 4 (avec shield), qui sert d'ordinateur principal pour la réalisation des acquisitions
- Composants utilitaires :
  - Plaque d'essai
  - Batterie externe
  - LED (pour le signalement d'une acquisition en cours)
  - Écran LCD (pour l'affichage des coordonnées GNSS)
  - Bouton poussoir (pour la gestion d'une acquisition)

D'autres composants étaient disponibles mais nous ne les avons pas utilisés durant notre projet car inutiles ou difficilement utilisables pour celui-ci.

2.2 Organisation logicielle

Pour ce projet nous avons découpé nos modules en différents dossiers et sous-dossiers afin de ne pas mélanger le code et les tests. Voici un détail de notre architecture de dossiers et des commentaires expliquant le regroupement :

```
projet/
|-- code/                       | Répertoire contenant tout le code
|   |-- config/                 | Fichiers de configuration
|   |-- core/                   |
|   |-- data/                   | Destination des acquisitions
|   |   `-- zip_acquisition/    | Acquitions réalisées en .zip
|   |-- postprocessing/         |
|   |   |-- plot/               | Tous les fichiers de plot (GNSS, IMU)
|   |   |-- preprocessing/      | Module filtrage des données
|   |   |-- squat/              | Module fonctionnalité supplémentaire
|   |   `-- fichiers .py/       | Fichiers .py dont le filtre de Kalman
|   |-- scripts/                | Scripts d'acquisition
|   |-- sensors/                |
|   |   |-- GNSS/               |
|   |   `-- IMU/                |
|   |-- serveur_web/            | Module web de la partie 2
|   `-- utils/                  |
`-- README.md                   | README du projet
```

2.3 Réalisation d'une acquisition

Pour réaliser des acquisitions, nous passons par de multiples étapes que nous détaillerons ensuite.

— Calibration (réalisée une seule fois)
— Acquisition et sauvegarde des données
— Prétraitement et filtrage des données
— Application du filtre de Kalman
— Application de notre méthode de notation et visualisation des résultats

3 Partie 1 : Estimation de la trajectoire

Dans cette partie, nous détaillerons les choix techniques réalisés ainsi que la manière dont nous avons obtenu des résultats fiables de notre système de capteurs.

3.1 Problématiques de la partie 1

Afin d'obtenir des résultats satisfaisants, nous avons dû manipuler des données brutes avec des qualités variables.

Le GPS nous fournit des données plutôt précises concernant notre position, cependant il n'est pas toujours fonctionnel car il dépend des signaux satellites qu'il reçoit. En extérieur, cela ne pose aucun problème mais dans les bâtiments, on a un taux d'erreur qui peut exploser.

L'IMU, quant à elle, des données en permanence sur notre accélération, notre vitesse de rotation et des données concernant le champ magnétique. On pourrait penser qu'il suffirait d'utiliser l'IMU en permanence afin de reconstruire notre trajet. Mais c'est impossible à cause d'un problème intrinsèque, la divergence. En effet, les données peuvent être utiles afin de recomposer un trajet, mais elles sont trop sensibles au bruit et aux biais lorsqu'elles sont utilisées seules. En utilisant uniquement l'IMU notre position n'a aucun sens car les intégrales nécessaires au calcul de position font diverger les valeurs trop rapidement.

Ainsi, on doit trouver un équilibre entre l'usage du GPS et de l'IMU. Par exemple, le GPS comme référence en extérieur, et l'IMU pour assurer une estimation plus continue entre deux mesures GPS.

Un exemple classique de problème du GPS est l'apparition ponctuelle de valeurs aberrantes, une position qui varie de plus de 1 km en une seconde. Cela correspond à une vitesse de 3600 km/h ou presque trois fois la vitesse du son.

3.2 Capteurs retenus

Ainsi pour notre fusion de capteurs, nous avons réalisé une fusion entre l'IMU et le GPS.

L'IMU pour les données à haute fréquence et la correction des données GNSS. La fréquence retenue est de $52Hz$ car elle est largement supérieure à la fréquence de marche. Pour le magnétomètre, on a retenu $40Hz$ pour être proche de la fréquence de l'accéléromètre et du gyroscope.

Pour l'IMU nous faisons l'acquisition de :

— Accel_{X,Y,Z}, l'accélération selon les axes de l'IMU

— Gyro_{X,Y,Z}, la vitesse de rotation selon les axes de l'IMU

— Magneto_{X,Y,Z}, les composantes du champ magnétique selon les axes de l'IMU

Le GPS pour la position de référence. Pour le GPS, nous faisons l'acquisition de :

— Latitude

— Longitude

— Altitude

— PDOP, HDOP, VDOP, la précision estimée des données

— COG_Magnetic, estimation de l'orientation à partir des données, malheureusement, nous n'avons aucune donnée des satellites

3.3 Calibration des capteurs

Pour manipuler correctement les données obtenues, nous devions effectuer une calibration de nos capteurs.

Pour cela, nous avons réalisé une acquisition statique (`acquisition_static.zip`) afin d'obtenir les biais des capteurs. Ces données ne sont pas utilisées durant l'acquisition mais seulement en post-traitement car il était plus simple de traiter les données à ce moment-là en tenant compte des biais. Les données de l'IMU servent pour le filtre de Kalman directement dans les matrices R et Q. Et la confiance accordée aux mesures GPS dans le filtre.

3.4 Script d'acquisition

Afin de réaliser une acquisition, nous exécutons le script :
`scripts/capture_gnss_imu_magnetometre.py`

3.5 Pré-traitement et filtrage des données brutes

Deux filtrages importants sont réalisés dans le cadre de ce projet :

— Filtrage des données aberrantes du GPS

— Filtrage des doublons dans nos données IMU et application de la correction statique.

3.5.1 Filtrage des données GPS

Au début du projet, le GPS produisait parfois des erreurs de lecture des données GNSS et nous fournissait des points totalement aberrants, avec des erreurs allant de plusieurs mètres à plusieurs milliers de kilomètres. Nous avons donc produit un code (`postprocessing/filter_gps.py`) qui prend en paramètre la distance seuil et les fichiers de données de GNSS avant et après filtrage.

L'objectif de ce programme est d'enlever les points qui sont à une distance seuil du dernier point valide reconnu. Pour cela, il fonctionne en 3 étapes :

— Détection du premier point valide. Un point est considéré comme valide si lui-même et les $X - 1$ points suivants sont à une distance inférieure à une valeur donnée Y (en mètres). X et Y étant des données modifiables dans le programme
Par défaut, $X = 3$ et $Y =$ donné par l'utilisateur.

— À partir du premier point valide, on vérifie si le point actuel est à une distance $Z \leq Y$.

— Si le point est valide alors il devient le dernier point valide et on continue, sinon on l'ignore.

Le choix de Y que nous faisons est de 30m car un point à une distance supérieure à 30m indiquerait un déplacement à plus de 108 km/h.

On peut appeler le programme de la manière suivante :

```
python postprocessing/filter_gps.py --max_distance 30 in.csv out.csv
```

On utilise la formule de Haversine pour le calcul de la distance entre deux points.

3.5.2 Nettoyage des données

Nos données IMU ont un problème lié à la récupération des données en I2C. L'IMU ne renvoie pas de signal lorsqu'elle fait une nouvelle acquisition de données, on doit donc faire une boucle qui attend selon la fréquence définie avant de récupérer les données. Cependant, cette boucle ne vérifie pas si les données ont changé ou non. On a donc des lignes en double dans notre fichier CSV de données IMU. Parfois des lignes entières, parfois des morceaux de la ligne seulement. Nous avons fait le choix de retirer les lignes entières en double lors du nettoyage des données avant de les fournir à notre filtre de Kalman, afin d'éviter des divergences et réduire le nombre de calculs. On remarque une diminution d'environ 20% du nombre de lignes dans notre fichier CSV ce qui n'est pas négligeable quand on a des acquisitions longues avec des dizaines de milliers de lignes.

Le code chargé de ce filtrage va aussi supprimer le biais des données de l'IMU et ajouter des colonnes contenant les données converties dans le DataFrame pandas retourné par la fonction. L'objectif est de donner à notre filtre de Kalman des données correctes ainsi que les données statiques associées à notre ensemble de capteurs afin de séparer la logique de traitement et la correction des biais.

On ajoute par exemple les données GPS par rapport au point d'origine et la conversion des données brutes du magnétomètre.

3.6 Modélisation du filtre de Kalman

3.6.1 Choix du modèle d'état

L'objectif du filtre de Kalman est d'estimer la trajectoire du porteur à partir des données GPS et IMU. Pour cela, nous avons choisi le vecteur d'état suivant :

$$X = \begin{bmatrix} x & y & v_x & v_y & \theta \end{bmatrix}^T$$

où x et y représentent la position dans un repère cartésien local, v_x et v_y les composantes de la vitesse selon les axes x et y, et θ l'orientation du système.

Ce choix est directement lié aux capteurs disponibles. Le GPS fournit une estimation de la position, tandis que l'IMU permet d'obtenir des informations sur les accélérations et les rotations. L'inclusion de la vitesse dans le vecteur d'état permet de prédire l'évolution de la position entre deux mesures GPS. L'orientation est également intégrée afin de pouvoir projeter les accélérations mesurées par l'IMU dans le repère du monde.

Avant l'application du filtre, les données GPS sont préalablement nettoyées afin de supprimer les mesures aberrantes susceptibles de perturber l'estimation.

3.6.2 Modèle de prédiction

La phase de prédiction repose exclusivement sur les mesures de l'IMU.

Les accélérations fournies par l'accéléromètre sont d'abord converties du repère de l'IMU vers le repère du monde à l'aide du cap estimé à partir du magnétomètre. Ces accélérations sont ensuite utilisées pour mettre à jour les composantes de vitesse et de position.

Le gyroscope fournit, quant à lui, la vitesse angulaire permettant de mettre à jour l'orientation du système.

À chaque itération, le filtre prédit donc la position, la vitesse et l'orientation à l'instant suivant à partir de l'état précédent et des mesures inertielles. Cette étape permet d'obtenir une estimation continue même lorsqu'aucune mesure GPS n'est disponible.

3.6.3 Modèle de mesure

La phase de correction utilise les mesures GPS et magnétométriques.

Les coordonnées GPS (latitude et longitude) sont converties en coordonnées cartésiennes afin d'être exprimées dans le même repère que le vecteur d'état. Ces mesures servent à corriger les composantes de position x et y.

Le magnétomètre est utilisé pour estimer le cap, associé ici à l'orientation θ. Cette mesure permet de corriger la dérive accumulée lors de l'intégration de la vitesse angulaire utilisée pour mettre à jour θ, améliorant ainsi la précision de l'estimation de l'orientation.

Ainsi :

— les données IMU (accéléromètre et gyroscope) sont utilisées pour la prédiction ;

— les données GPS corrigent la position ;

— le magnétomètre corrige l'orientation.

Cette organisation permet de bénéficier de la fréquence élevée de l'IMU tout en limitant la dérive grâce aux mesures absolues fournies par le GPS.

3.6.4 Matrices du filtre

La matrice de transition d'état utilisée est :

$$F = \begin{bmatrix} 1 & 0 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & \Delta t & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Cette matrice traduit un modèle cinématique où la position évolue en fonction de la vitesse entre deux instants successifs.

La matrice d'observation est :

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Elle indique que les mesures disponibles permettent d'observer directement la position (x, y) ainsi que l'orientation θ.

La matrice de bruit de processus est choisie diagonale :

$$Q = \begin{bmatrix} 0.1 & 0 & 0 & 0 & 0 \\ 0 & 0.1 & 0 & 0 & 0 \\ 0 & 0 & 0.1 & 0 & 0 \\ 0 & 0 & 0 & 0.1 & 0 \\ 0 & 0 & 0 & 0 & 0.1 \end{bmatrix}$$

Elle représente l'incertitude associée au modèle de prédiction et aux mesures inertielles utilisées pendant cette étape.

La matrice de bruit de mesure est également diagonale :

$$R = \begin{bmatrix} 5 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 0.1 \end{bmatrix}$$

Les deux premières composantes correspondent à l'incertitude associée aux mesures GPS de position, tandis que la troisième représente l'incertitude sur l'orientation estimée à partir du magnétomètre.

Le choix de ces valeurs résulte d'un compromis entre confiance accordée au modèle de prédiction et aux mesures. Les données GPS étant préalablement filtrées pour supprimer les valeurs aberrantes, elles peuvent être utilisées comme mesures de correction tout en conservant une certaine tolérance aux erreurs de localisation résiduelles.

3.7 Résultats obtenus

Nous avons fait différentes acquisitions avec des paramètres différents pour tester notre filtre. Les acquisitions sont disponibles dans notre dépôt Git. Ainsi nous avons exécuté un script `kalman.sh` qui permet d'appliquer nos filtres et qui renvoie des fichiers csv contenant les mêmes types de données que le fichier issu de l'acquisition GPS. Cela nous permet d'exécuter les fichiers de visualisation afin d'avoir les cartes et les comparaisons.

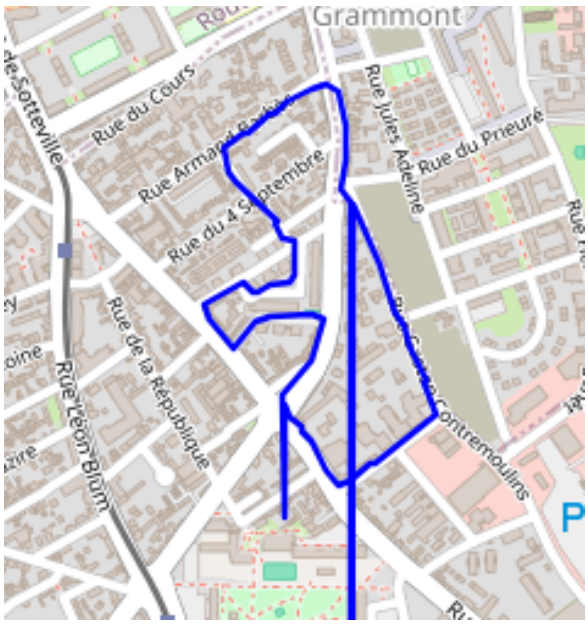


FIGURE 1 – Acquisition faite en marchant



FIGURE 2 – Acquisition faite en marchant et filtré

On a voulu tester jusqu'à quand notre filtre était efficace. Nous avons tout d'abord voulu tester l'effet d'une variation de vitesse. Ainsi, nous avons décidé de prendre une

acquisition en métro. Pendant l'acquisition, le GPS n'a pas perdu le signal et l'acquisition GPS était satisfaisante. Nous avons appliqué notre filtre. Ce dernier a juste légèrement lissé la courbe. Nous en avons déduit que notre Kalman n'était pas forcément faux mais qu'il pouvait être remis en cause.

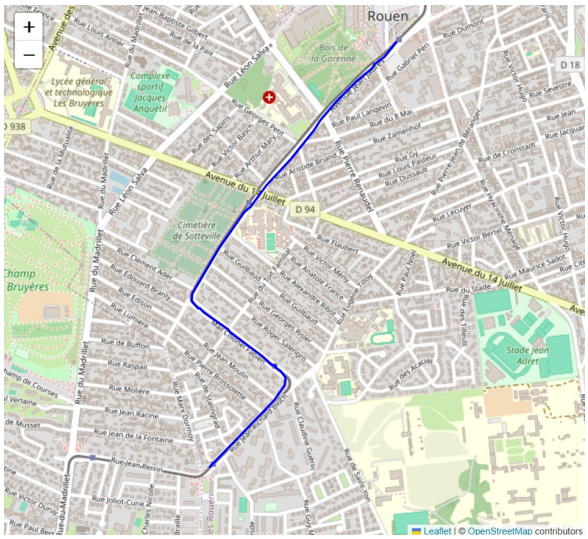


FIGURE 3 – Acquisition faite dans le métro

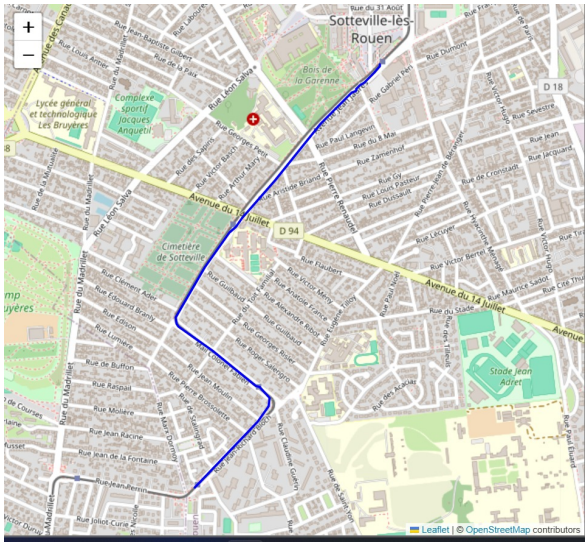


FIGURE 4 – Acquisition faite dans le métro filtrée

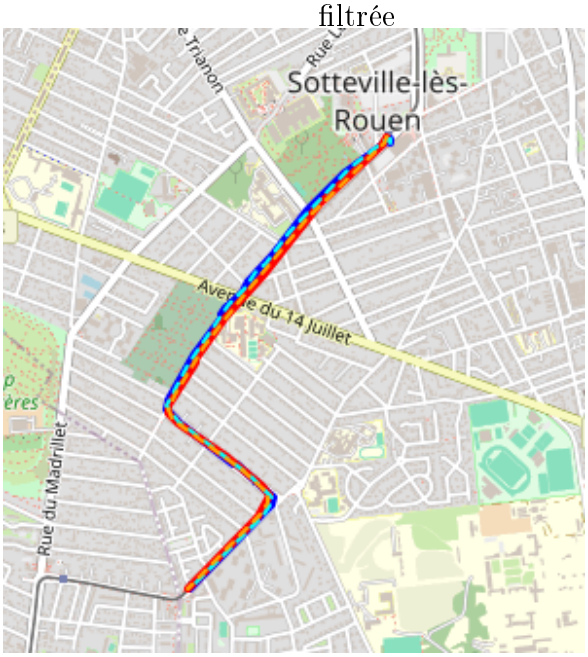


FIGURE 5 – Application d'une fonction de comparaison entre l'acquisition filtrée et non filtrée

Nous avons fait une acquisition par temps couvert pour voir si cela avait un impact sur nos acquisitions. Malgré cela, le GPS a bien capté et l'acquisition était très proche de la réalité. Ces essais ont montré qu'il était difficile de montrer si notre filtre de Kalman était efficace.

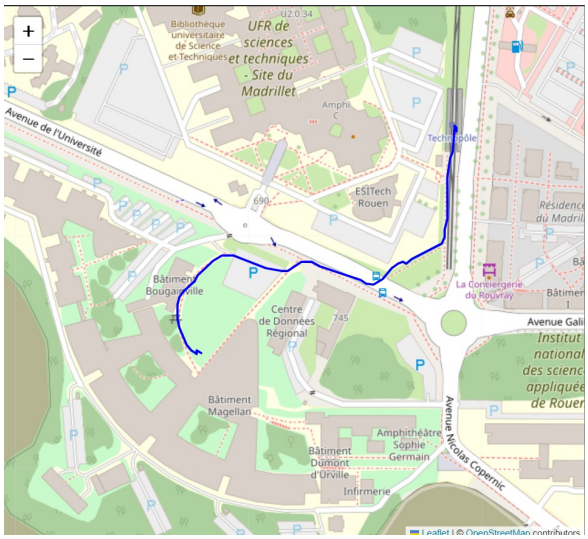


FIGURE 6 – Acquisition faite sous les nuages

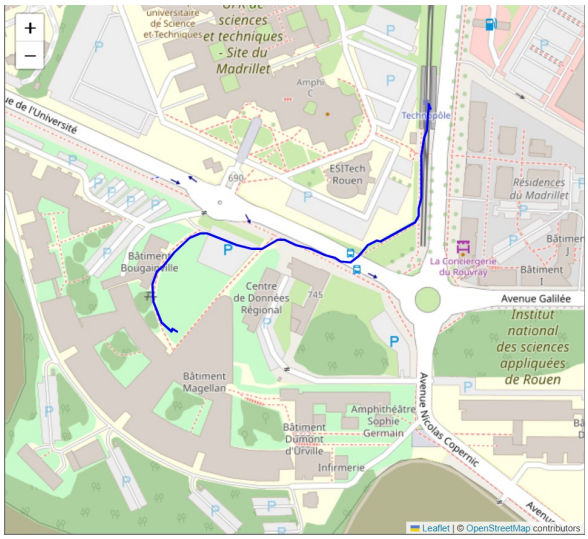


FIGURE 7 – Acquisition faite sous les nuages filtré

Ainsi, nous avons décidé de faire une grande acquisition afin de perturber notre GPS. Nous sommes rentrés dans les bâtiments de l'INSA et alterné entre perte de signal GPS et reprise de signal GPS durant 10 minutes pour tester le comportement de notre filtre en cas de perte de signal.

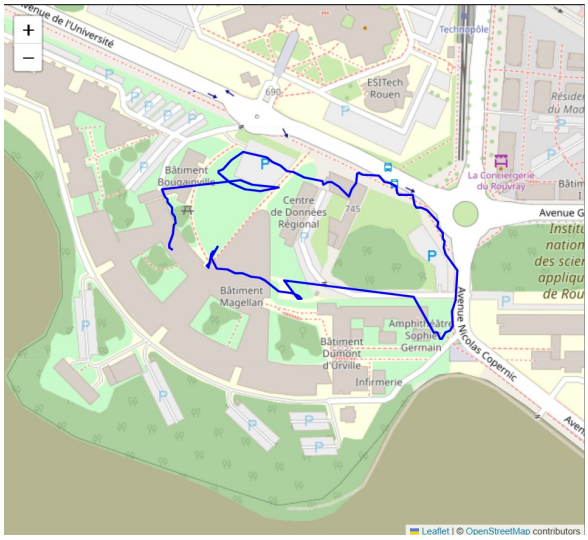


FIGURE 8 – Acquisition avec perte de signal GPS

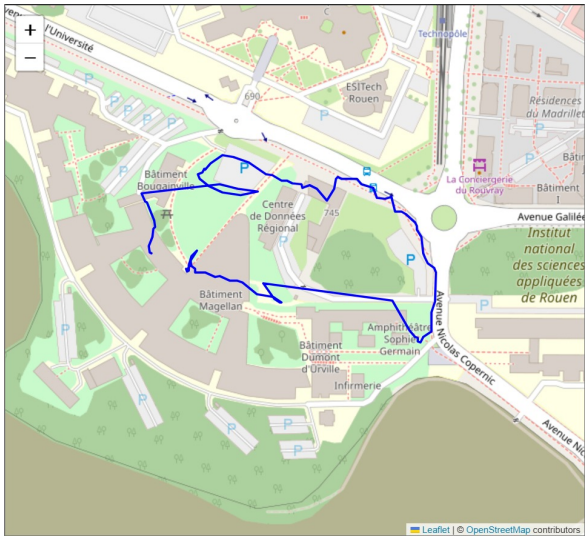


FIGURE 9 – Acquisition filtrée avec perte de signal GPS

Dans cette image nous pouvons voir que lorsque le GPS ne capte pas de signal, il y a quelques points aberrants et l'algorithme les relie de manière linéaire. Ainsi, des lignes droites sont déterminées de manière aléatoire lorsque nous affichons nos valeurs. En appliquant notre filtre nous avons pu voir que la différence entre avant et après le filtre

de Kalman était sensiblement la même. Seulement sur les virages, les trajectoires sont modifiées. Elles sont lissées mais il n'y a pas de modification majeure de la trajectoire. Cela confirme que le filtre se base principalement sur le GPS mais que l'apport de l'IMU modifie peu la trajectoire. Nous avons tenter d'augmenter la confiance de l'imu mais il y a eu de nombreux points répartis aléatoirement ce qui n'est pas exploitable et ne renvoie pas la bonne direction.

Notre filtre de Kalman dépend de la qualité des données du GPS et l'IMU permet de corriger les petits écarts. Lors de nos premières acquisitions, le GPS avait des points assez proches de la réalité. Ce qui rendait le filtre efficace. Malheureusement, le filtre ne corrige pas correctement les valeurs du GPS quand le signal est perdu longtemps.

3.8 Conclusion partie 1

Pour conclure, Nous avons réalisé à l'aide du GPS et de l'IMU un ITIPhone capable d'estimer la trajectoire de son utilisateur. Pour cela il effectue une fusion des données entre le GPS et l'IMU. Nous avons réalisé un premier filtrage qui enlève les valeurs aberrantes puis un filtre de Kalman qui dépend de la qualité des données du GPS. Un axe d'amélioration serait peut-être d'ajouter d'autres capteurs pour avoir plus de données, améliorer la précision de nos acquisitions et réduire la dépendance au GPS.

4 Partie 2 : Fonctionnalité supplémentaire

4.1 Présentation de la fonctionnalité choisie

La fonctionnalité supplémentaire développée dans le cadre de ce projet est un compteur automatique de squats, exploitant les données inertielles fournies par l'IMU embarquée sur l'ITIPhone. L'objectif est de détecter et de comptabiliser les squats effectués par l'utilisateur à partir d'une session d'acquisition préalablement enregistrée.

Un squat est un mouvement physique cyclique et répétitif : l'utilisateur descend en fléchissant les genoux, puis remonte à sa position initiale. Ce cycle de descente et de remontée génère une signature caractéristique dans les signaux d'accélération, ce qui facilite le traitement du signal et en fait un cas d'usage particulièrement adapté à l'analyse des signaux inertiels.

Le fonctionnement de notre application repose sur deux étapes distinctes. Dans un premier temps, une phase d'acquisition est réalisée : l'ITIPhone, porté par l'utilisateur durant sa session sportive, enregistre les données brutes de l'IMU (accéléromètre et gyroscope) sur la Raspberry Pi embarquée. Dans un second temps, un programme de traitement analyse ces données en post-traitement afin de filtrer le signal, de détecter les cycles cinématiques et de comptabiliser précisément les répétitions valides.

4.2 Cahier des charges de la fonctionnalité

Afin de cadrer le développement de cette preuve de concept, un cahier des charges minimal a été défini. Il fixe les exigences auxquelles le système doit répondre, aussi bien en termes d'acquisition des données que de traitement et de restitution des résultats.

Les exigences fonctionnelles et techniques retenues sont les suivantes :

— *Acquisition des données inertielles* : le système doit enregistrer en continu, durant toute la session sportive, les données de l'accéléromètre et du gyroscope de l'IMU embarquée, horodatées et stockées localement sur la Raspberry Pi.

— *Traitement du signal en amont* : les données brutes ne doivent pas être utilisées directement pour la détection. Un filtrage est appliqué afin d'éliminer le bruit à haute fréquence et de ne conserver que la composante significative du mouvement.

— *Détection automatique des répétitions* : le programme de post-traitement doit identifier chaque cycle complet et incrémenter un compteur à chaque squat validé.

— *Robustesse aux faux positifs* : le système doit distinguer un squat complet d'un mouvement parasite ou d'une répétition incomplète, en combinant les informations de l'accéléromètre et du gyroscope pour fiabiliser la détection.

4.3 Choix des capteurs

La détection des squats repose sur deux capteurs de l'IMU embarquée sur l'ITIPhone : l'accéléromètre et le gyroscope.

4.3.1 Accéléromètre

L'accéléromètre constitue le capteur principal de cette fonctionnalité. Il mesure les accélérations propres du dispositif selon ses trois axes orthogonaux. Lors d'un squat, le mouvement vertical du corps de l'utilisateur se traduit par des variations périodiques et

caractéristiques de l'accélération : une phase d'allègement lors de la descente, suivie d'une phase de poussée lors de la remontée. C'est cette signature cyclique que le système exploite pour identifier et comptabiliser chaque répétition.

4.3.2 Gyroscope

Le gyroscope intervient en complément en mesurant la vitesse angulaire du dispositif durant l'effort. Un squat correctement exécuté implique non seulement une translation verticale, mais aussi une variation d'inclinaison de la cuisse ou du tronc, selon la position du téléphone. Le gyroscope permet ainsi d'apporter un critère de validation supplémentaire : une répétition n'est validée que si les deux capteurs détectent simultanément un mouvement cohérent, ce qui réduit significativement les faux positifs liés à des gestes parasites.

4.4 Architecture logicielle

4.4.1 Arborescence des fichiers

```
code/                          | Répertoire contenant tout le code
|-- data/                      | Destination des acquisitions
|   `-- acquisition_squat/     | Fichiers d'acquisition
|       `-- parameters.py      | Fichier .py avec paramètres
|       `-- timestamps_squat   | Fichier .csv avec données
|-- postprocessing/            |
|   `-- squat/                 | Module fonctionnalité squat
|       |-- analyse_squat.py   | Fichier .py traitement données
|       `-- process_squat.py   | Analyseur de squats
`-- serveur_web/               | Module web
```

4.4.2 Structuration du fichier `timestamps_squat.csv`

Chaque ligne du fichier correspond à un relevé de mesure et est composée de huit colonnes, regroupées comme suit :

- `Index` : un identifiant entier séquentiel, incrémenté à chaque nouvelle mesure. Il permet de s'assurer de l'intégrité de l'enregistrement et de l'absence de pertes de données.
- `Timestamp` : l'horodatage de l'acquisition. Cette colonne est essentielle car elle permet de calculer la période d'échantillonnage effective (Δt entre deux mesures) pour paramétrer les filtres numériques, et de replacer les mouvements selon une chronologie précise.
- `Accel_X`, `Accel_Y`, `Accel_Z` : les trois composantes spatiales du vecteur d'accélération linéaire, exprimées en multiples de la gravité terrestre (g). Ces données alimentent le calcul de la norme vectorielle.
- `Gyro_X`, `Gyro_Y`, `Gyro_Z` : les trois composantes de la vitesse angulaire mesurées par le gyroscope, exprimées en degrés par seconde (deg /s). Elles sont utilisées pour vérifier l'amplitude de la rotation articulaire lors du mouvement.

4.5 Traitement des données

La transformation des données brutes issues de l'IMU en informations exploitables nécessite une chaîne de traitement fiable et efficace. Les signaux bruts sont en effet sujets au bruit électronique, aux vibrations mécaniques et aux changements d'orientation spatiale du capteur. Pour y répondre, notre chaîne de traitement applique deux opérations successives : un calcul de norme vectorielle et un filtrage fréquentiel.

4.5.1 Calcul de la norme vectorielle

Le capteur étant positionné de manière variable sur l'utilisateur (dans une poche, fixé sur la cuisse ou tenu dans les mains), le vecteur de gravité terrestre se répartit de façon changeante sur les axes X, Y et Z. Analyser un axe isolé conduirait inévitablement à des erreurs d'interprétation lors des changements de posture.

Pour s'affranchir de l'orientation du boîtier, nous calculons la norme vectorielle euclidienne de l'accélération à chaque instant t :

$$\|A_t\| = \sqrt{A_{x,t}^2 + A_{y,t}^2 + A_{z,t}^2} \tag{1}$$

Indépendamment de l'inclinaison du capteur, la norme $\|A\|$ converge vers $1\,g$ au repos et reflète fidèlement l'intensité de la poussée. Cette opération est implémentée dans la fonction `normalize` :

```python
def normalize(acc_x: np.ndarray, acc_y: np.ndarray, acc_z: np.ndarray,
              gyr_x: np.ndarray, gyr_y: np.ndarray, gyr_z: np.ndarray):
    """
    Normalise les donnees de l'accelerometre et du gyroscope.
    Retourne: tuple (accel_norm, gyro_norm) sous forme de tableaux numpy.
    """
    accel_norm = np.sqrt(acc_x**2 + acc_y**2 + acc_z**2)
    gyro_norm  = np.sqrt(gyr_x**2 + gyr_y**2 + gyr_z**2)

    return accel_norm, gyro_norm
```

Listing 1 – Fonction de calcul des normalisation vectorielle.

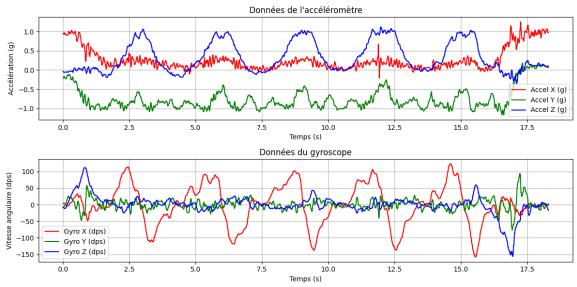


FIGURE 10 – Norme brute de l'accélération $\|A\|$ avant normalisation.

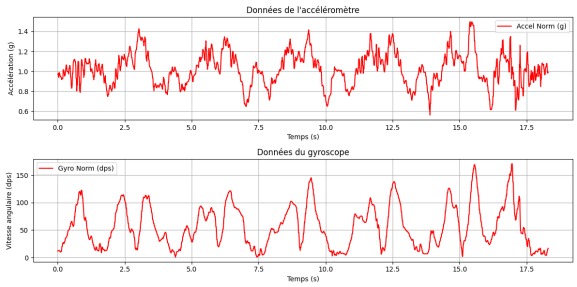


FIGURE 11 – Norme de l'accélération $\|A\|$ après calcul de la norme vectorielle.

4.5.2 Filtrage numérique passe-bas

Les signaux inertiels contiennent une composante à haute fréquence indésirable due aux bruits de mesure et aux micro-vibrations musculaires. Le mouvement d'un squat étant

par nature lent (durée comprise entre 1 et 4 secondes, soit une fréquence fondamentale d'environ 0,25 à 1 Hz), l'application d'un filtre est indispensable.

Un filtre passe-bas de Butterworth d'ordre 4 a été retenu. Ce choix se justifie par sa réponse en fréquence aussi plate que possible dans la bande passante, évitant ainsi d'altérer l'amplitude des pics d'accélération utiles. La fréquence de coupure a été fixée expérimentalement à $f_c = 2{,}0\,\text{Hz}$.

L'application de ce filtre est réalisée via la fonction `filtfilt` de la bibliothèque SciPy. Contrairement à un filtrage causal classique, `filtfilt` effectue un passage aller-retour sur le signal, ce qui permet d'obtenir un filtrage à phase nulle et de conserver un bon alignement temporel des événements physiques.

```python
def filter_signal(raw_data: np.ndarray, fs: float = 50.0,
                  cutoff: float = 2.0, order: int = 4) -> np.ndarray:
    """
    Methode permettant de filtrer les donnees avec un filtre de Butterworth (passe-bas
    ).
    """
    nyq = fs / 2.0
    b, a = butter(order, cutoff / nyq, btype='low', analog=False)

    # filtfilt garantit l'absence de dephasage
    return filtfilt(b, a, raw_data)
```

Listing 2 – Fonction de filtrage de Butterworth à phase nulle.

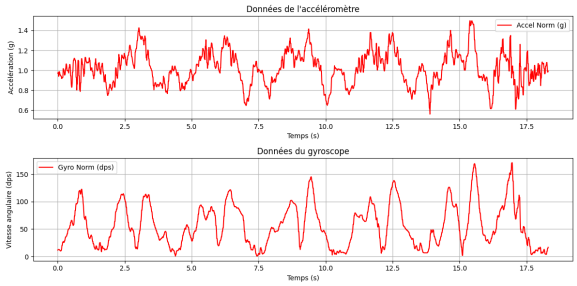


FIGURE 12 – Signal brut de $\|A\|$ avec bruit à haute fréquence.

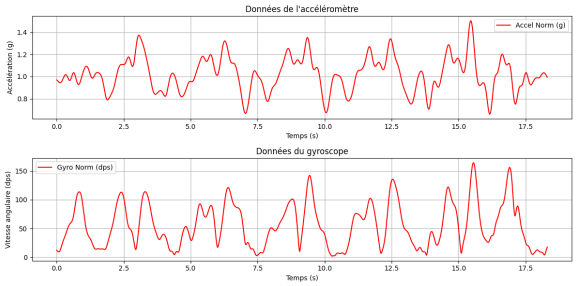


FIGURE 13 – Signal de $\|A\|$ après application du filtre de Butterworth.

4.6 Analyse des données

4.6.1 Paramétrage de l'algorithme de détection

L'algorithme de détection repose sur six constantes calibrées de manière empirique, regroupées en trois familles fonctionnelles. Le choix de ces valeurs résulte d'itérations successives sur les données acquises, en cherchant à maximiser le taux de détection tout en minimisant les faux positifs.

Seuils accélérométriques

Les trois seuils suivants opèrent sur la norme filtrée du vecteur d'accélération, exprimée en multiples de g :

— `SEUIL_DESCENTE` $= 0{,}90\,g$: en dessous de ce seuil, le signal traduit un allègement de la charge sur le capteur, caractéristique de la phase de descente du squat. La valeur

de 0,90 g offre une marge de 0,10 g sous la gravité de repos, suffisante pour absorber le bruit résiduel après filtrage sans déclencher sur de simples oscillations posturales.

— `SEUIL_POUSSEE` = 1,05 g : dépassement transitoire de la valeur au repos, induit par l'impulsion musculaire concentrique lors de la remontée. Fixer ce seuil à 1,05 g permet de ne déclencher que sur une poussée franche, tout en restant accessible à des sujets produisant une impulsion modérée.

— `SEUIL_REPOS` = 1,05 g : ce seuil correspond au retour sous cette même valeur après la poussée, ce qui indique la stabilisation posturale et permettant de clore l'événement détecté. L'alignement de cette valeur avec `SEUIL_POUSSEE` garantit une transition nette entre la phase active et le retour au repos.

Contraintes temporelles

— `TEMPS_MIN_SQUAT` = 1,25 s : durée minimale requise pour qu'un mouvement soit considéré comme valide. Ce seuil écarte les impacts et micro-perturbations dont la signature accélérométrique peut franchir les seuils sans correspondre à un squat complet. En pratique, un squat correctement exécuté dure rarement moins d'une seconde, ce qui confère une marge de sécurité de 0,25 s.

— `TEMPS_MAX_SQUAT` = 4,0 s : garde temporelle haute au-delà de laquelle la séquence est invalidée. Cette valeur prévient le blocage de l'automate sur un signal lentement variable - dérive de capteur ou changement de posture prolongé - qui ne présenterait jamais de pic de poussée franc.

Seuil gyroscopique

— `SEUIL_ROTATION` = 40,0 deg /s : vitesse angulaire minimale devant être observée dans la fenêtre temporelle du mouvement, utilisé uniquement lorsque l'utilisation du gyroscope est activée. Ce critère apporte une dimension qualitative à la détection : là où l'accéléromètre seul ne peut distinguer un squat complet d'un déséquilibre vertical, le gyroscope atteste d'une rotation effective du dispositif, cohérente avec le mouvement attendu, réduisant significativement le risque de faux positifs.

4.6.2 Machine à états finis : logique de détection en trois phases

La détection est implémentée sous la forme d'une machine à états finis à trois états. À chaque échantillon i, la valeur de la norme filtrée a_i est comparée aux seuils définis pour mettre à jour la variable d'état `etat` $\in \{0, 1, 2\}$.

État 0 - Veille et détection de l'amorce

L'automate réside dans cet état au démarrage et après chaque cycle de traitement. Il surveille en continu le signal en attente d'une transition descendante. Dès que $a_i <$ `SEUIL_DESCENTE`, l'indice courant i et l'horodatage t_i sont mémorisés dans `idx_debut_descente` et `temps_debut_descente`, puis l'automate passe en état 1. Ces deux grandeurs constituent le point de départ temporel du cycle de validation.

État 1 - Suivi de la phase descendante et attente de l'impulsion

L'automate reste en état 1 tant que le signal n'a pas encore dépassé `SEUIL_POUSSEE`. Deux transitions sont possibles :

— si $a_i >$ `SEUIL_POUSSEE`, l'impulsion concentrique est détectée et l'automate passe en état 2 ;

— si ($t_i -$ `temps_debut_descente`) $>$ `TEMPS_MAX_SQUAT`, la durée maximale est dépassée sans qu'aucune poussée n'ait été observée ; le mouvement est abandonné et l'automate revient en état 0.

Ce second branchement constitue une limite temporelle haute : il empêche l'automate de rester bloqué en état 1 sur un signal lentement variable ne présentant jamais de pic de poussée.

État 2 - Clôture du mouvement et réarmement

Après la détection de la poussée, l'automate attend le retour du signal en dessous de `SEUIL_REPOS`. Lorsque $a_i <$ `SEUIL_REPOS`, l'instant de fin t_{fin} est enregistré, la durée totale $\Delta t = t_{\text{fin}} -$ `temps_debut_descente` est calculée, et le bloc de validation est déclenché. L'automate est systématiquement réinitialisé à l'état 0 à l'issue de ce bloc, quelle que soit l'issue de la validation, ce qui permet le réarmement pour le mouvement suivant.

4.6.3 Validation multicritère du mouvement détecté

Le franchissement des trois états de l'automate ne suffit pas à valider un squat. Deux vérifications complémentaires sont appliquées séquentiellement avant tout enregistrement.

Vérification 1 - Durée minimale

La durée Δt est comparée au seuil `TEMPS_MIN_SQUAT`. Si $\Delta t < 1{,}25\,\text{s}$, le mouvement est rejeté et un message horodaté est affiché :

`[Refusé] Mouvement à {t}s rejeté : Trop rapide ({`Δt`}s).`

Ce filtre élimine les impulsions parasites d'amplitude élevée – chocs, repositionnements brusques – dont la signature temporelle aurait pu franchir les seuils accélérométriques sans correspondre à un mouvement réel de squat.

Vérification 2 - Cohérence gyroscopique (optionnelle)

Lorsque `use_gyro = True`, une seconde condition est évaluée après la vérification temporelle. La sous-séquence `gyro_norm_filt[idx_debut_descente:i]` est extraite par découpage indiciel, isolant strictement les données gyroscopiques correspondant à la fenêtre du mouvement. Elle est transmise à la fonction `valider_rotation_gyro()`, chargée de vérifier si la vitesse angulaire observée dépasse `SEUIL_ROTATION`. En cas d'échec, le mouvement est rejeté :

`[Refusé] Mouvement à {t}s rejeté : Manque de rotation (Flexion insuffisante).`

L'apport de cette modalité est de nature qualitative : elle permet de discriminer un squat complet d'un déséquilibre vertical isolé, dont le profil accélérométrique serait similaire, mais avec une cinématique angulaire trop faible.

Un mouvement ayant satisfait tous les critères applicables est ajouté à la liste `squats_detectes` par son indice de fin i. Ce choix ancre chaque événement au moment de la stabilisation posturale, facilitant toute analyse ultérieure portant sur les intervalles entre répétitions.

4.7 Résultats

Cette section présente l'évaluation quantitative de l'algorithme de détection sur les sessions d'acquisition réelles réalisées avec l'ITIPhone. Cinq sessions distinctes ont été enregistrées, faisant varier la position du capteur, le rythme d'exécution et la nature du mouvement, afin de couvrir un spectre de conditions d'usage représentatif.

4.7.1 Protocole d'évaluation et métriques de performance

Pour évaluer la précision du compteur automatique, nous nous appuyons sur trois métriques standard du traitement du signal :

— *Vrais Positifs (VP)* : squats correctement exécutés et comptabilisés par l'algorithme.
— *Faux Positifs (FP)* : mouvements parasites comptabilisés à tort comme des squats.
— *Faux Négatifs (FN)* : squats réellement effectués mais non détectés par l'algorithme.

À partir de ces variables, la Précision (P) et la Sensibilité (S) sont calculées selon :

$$P = \frac{VP}{VP + FP} \qquad \text{et} \qquad S = \frac{VP}{VP + FN} \tag{2}$$

La précision reflète la capacité de l'algorithme à ignorer les perturbations, tandis que la sensibilité mesure sa capacité à ne pas manquer les répétitions réellement effectuées.

En l'absence d'horodatage de référence pour chaque squat réel, le nombre de VP est estimé par hypothèse optimiste : on suppose que toute détection correspond à un squat réel, dans la limite du nombre de squats effectivement réalisés, soit $VP = \min(\text{détectés, réels})$. La Précision est donc évaluée au mieux, tandis que la Sensibilité, qui repose uniquement sur le comptage réel, reste une reste la métrique la plus fiable dans ce protocole.

4.7.2 Analyse globale des performances

Le système a été évalué sur cinq sessions d'acquisition totalisant 21 squats réels. Deux positions de port du capteur ont été testées : la plaque tenue devant l'utilisateur, et la plaque fixée sur le côté droit de la cuisse en suivant le mouvement de la jambe. Pour les sessions en position latérale, la validation gyroscopique est activée ; pour les sessions avec la plaque tenue devant, elle est désactivée, la rotation articulaire étant peu transmise au capteur dans cette configuration. Les résultats sont synthétisés dans le Tableau 1.

Session	**Réels**	**VP**	**FP**	**FN**	**Précision**	**Sensibilité**
Squat régulier (plaque devant)	5	5	0	0	100,0 %	100,0 %
Squat régulier (position latérale)	5	5	0	0	100,0 %	100,0 %
Squat régulier + rapide (devant)	6	5	0	1	100,0 %	83,3 %
Squat rapide (position latérale)	5	4	0	1	100,0 %	80,0 %
Marche seule (0 squat)	0	0	0	0	100,0 %	100,0 %
Total	**21**	**19**	**0**	**2**	**100,0 %**	**90,5 %**

TABLE 1 – Synthèse des performances de détection par session d'acquisition.

Les résultats attestent d'une Précision estimée à 100 % sur l'ensemble des sessions : sous l'hypothèse optimiste décrite ci-dessus, aucun faux positif n'a été comptabilisé, y compris lors de la session de marche. La Sensibilité globale atteint 90,5 %, avec deux faux négatifs concentrés sur les sessions à rythme rapide. Ces manques s'expliquent par

le dépassement occasionnel de la garde temporelle haute (`TEMPS_MAX_SQUAT`) ou, pour la session latérale rapide, par un signal de pousée trop bref pour franchir `SEUIL_POUSSEE` avant que le signal ne se stabilise.

Un point notable concerne la position de port du capteur : les sessions avec la plaque tenue à bout de bras présentent une vitesse angulaire gyroscopique maximale comprise entre 14 et 28 deg/s, très inférieure au seuil de 40 deg/s. Dans cette configuration, la rotation articulaire des genoux et des hanches n'est pas transmise au capteur, ce qui rend la validation gyroscopique inopérante et impose de la désactiver. À l'inverse, les sessions en position latérale produisent des pics gyroscopiques allant jusqu'à 150 deg/s, pleinement compatibles avec le critère de détection.

4.7.3 Visualisations des sessions d'acquisition

Les trois configurations testées sont illustrées ci-dessous :

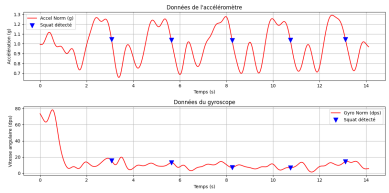


FIGURE 14 – Position plaque tenue devant soi.

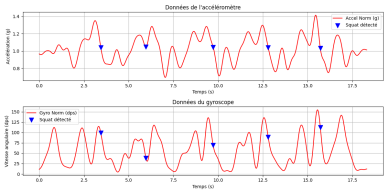


FIGURE 15 – Position plaque fixée sur le côté.

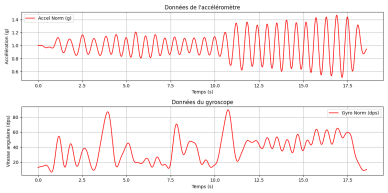


FIGURE 16 – Session de marche (contrôle).

4.7.4 Impact des filtres de robustesse : étude comparative

Pour mesurer l'apport concret de chaque couche de filtrage, une analyse comparative a été conduite en faisant varier le niveau d'activation des filtres de robustesse. Le test combine la session de marche (aucun squat réel) et la session de squat rapide en position latérale (5 squats réels), soit un total de 5 squats attendus sur un signal comportant une activité quotidienne variée.

Configuration de l'algorithme	Nombre de squats détectés	Faux positifs
Machine à états seule	37	32
Machine à états + Contrainte temporelle	4	0
Algorithme complet	4	0

TABLE 2 – Évolution du nombre de fausses détections selon le niveau d'activation des filtres.

Les résultats illustrent l'apport décisif de chaque filtre. La machine à états opérant sur les seuls seuils accélérométriques génère 32 faux positifs, la marche produisant des variations de norme suffisantes pour franchir les seuils de descente et de poussée. L'ajout de la contrainte temporelle élimine ces faux positifs, les chocs et les pas de marche étant par nature des événements brefs. La validation gyroscopique n'apporte pas de gain supplémentaire sur ce jeu de données, le nombre de squats détectés restant stable à 4 ; elle constituerait néanmoins un filtre complémentaire pertinent dans un contexte d'acquisition plus bruité ou avec une position de capteur moins favorable.

4.8 Perspectives d'amélioration

Plusieurs axes d'amélioration ont été identifiés à l'issue du développement de cette partie.

4.8.1 Vers un traitement en temps réel

Le traitement actuellement mis en place repose sur la fonction `filtfilt` de SciPy. Cette approche, dite non causale, est particulièrement avantageuse en post-traitement car elle garantit un déphasage nul et conserve un bon alignement temporel des événements physiques. Cependant, elle est par définition incompatible avec un traitement en temps réel : `filtfilt` nécessite de disposer de l'intégralité du signal, ce qui rend impossible son application sur un flux de données continu.

Pour envisager un fonctionnement en temps réel, il serait nécessaire de remplacer `filtfilt` par un filtre causal classique, appliqué échantillon par échantillon. De plus, le passage à un traitement en continu imposerait de repenser la logique de détection elle-même. Elle repose actuellement sur une analyse globale du signal enregistré, et devrait être adaptée à un traitement en continu.

4.8.2 Interface de visualisation

Dans l'optique d'améliorer l'expérience utilisateur, un serveur web développé en Python a été mis en place. Il est composé d'une interface permettant d'afficher le compteur de squats en temps réel. L'incrémentation du compteur est déclenchée par un simple appel à `/squat_detecte`, ce qui rend l'architecture modulaire et facilement extensible : le module de détection et l'interface d'affichage sont totalement découplés. À terme, cette interface pourrait servir à différents affichages comme : une visualisation du signal filtré, de statistiques de séance (cadence moyenne, durée des répétitions, etc) ou encore d'un historique des entraînements.

4.8.3 Robustesse de la détection

L'un des principaux défis de la détection de squats par signaux inertiels réside dans la discrimination entre un véritable squat et un mouvement parasite présentant un motif similaire. Sur des sessions longues, certains comportements du quotidien comme s'asseoir, ramasser un objet au sol ou s'accroupir brièvement peuvent générer des variations d'accélération comparables à celles d'un squat, et ainsi provoquer des faux positifs.

Le système actuel intègre déjà des contraintes temporelles sur la durée des répétitions ainsi qu'une validation croisée entre l'accéléromètre et le gyroscope. Cependant, ces mécanismes atteignent leurs limites sur de longues acquisitions ou dans des contextes de vie quotidienne. Pour aller plus loin, des approches par **apprentissage automatique** entraînées sur des séquences labellisées permettraient une robustesse supérieure à celle des méthodes à seuil fixe. Un modèle de classification serait capable d'apprendre le motif d'un squat plutôt que de se baser sur des critères heuristiques. Cela nécessiterait toutefois la constitution d'un jeu de données d'entraînement suffisamment varié, couvrant différents profils d'utilisateurs et différentes positions du capteur.

5 Partie bonus et acquisitions

Nous avons partagé nos données d'acquisition qui ont été réalisées dans des conditions variables.

On y retrouve notamment :

— Acquisition longue : d'une durée de 20 minutes sous un ciel nuageux

— Acquisition métro : trajet entre quelques stations de métro en métro

— Acquisition nuageux : acquisition sous les nuages et la pluie

— Acquisition statique : acquisition statique pour l'estimation des biais

Certaines acquisitions n'ont pas été fournies car elles contenaient des valeurs aberrantes ou des erreurs. L'une d'entre elles est une acquisition avec trois pertes de signal GNSS, car nous sommes passés à l'intérieur de plusieurs bâtiments. D'autres n'ont simplement pas de données Strava associées.

Nous avons tout de même utilisé ces données afin d'évaluer les limites de notre système, notamment en le cas de la perte de signal.

5.1 Conclusion

Ce projet nous a permis de développer nos connaissances sur l'utilisation des capteurs. Cela concerne surtout les acquisitions et les traitements multi-capteurs. Nous nous sommes globalement réparti le projet en deux parties. Le premier groupe (Hadi, Nicolas et Yann) a travaillé sur la partie 1. Le second groupe (Benoît et Maximilien) s'est chargé la création d'une fonctionnalité supplémentaire.

5.1.1 Matrice de contribution

Pour lire la matrice des contributions, on considère :
— O : Oui
— N : Non

Tâche	Assi	Grondin	Lesterlin	Rolla	Yvanoff
Partie 1					
Code d'acquisition	N	O	O	N	N
Acquisitions	O	O	O	O	O
Filtrage des données	N	O	N	N	N
Filtre de Kalman					
Conception	O	O	N	N	O
Implémentation	O	N	N	N	O
Partie 2					
Brainstorming	N	N	O	O	N
Implémentation	N	N	O	O	N
Rapport					
Rédaction	O	O	O	O	O
Forme & relecture	O	O	O	O	O

Table 3 – Matrice de contribution des membres du projet.